

EXPERIENCE

- Google DeepMind** Mountain View, CA
Research Engineer Jun 2025–Present
 - XLA Memory Timeline Visualizer:** Built an interactive XLA memory visualization tool in D3.js for inspecting the compiler's static buffer assignment and HBM layout. Parsed XLA buffer assignments and live range text dumps to visualize allocation lifetimes, dependencies across memory spaces, and physical layout/fragmentation.
 - Sparse Logit Computation:** Implemented a training memory optimization that reduces the decoded logit tensor from $[\text{seq_len}, \text{vocab_size}]$ to $[K, \text{vocab_size}]$ where K is the maximum number of trainable tokens per sequence. Uses `jax.lax.top_k` to identify trainable hidden states before the final projection, avoiding materialization of non-trainable logits.
 - Nested Activation Checkpointing and Offloading:** Designed a two-level activation memory strategy for Gemma 4: block-level rematerialization at attention and FFN boundaries composed with decoder-level input offloading to pinned host memory. More HBM-efficient than decoder-level rematerialization alone. Applied externally via module-tree traversal and dynamic subclassing, avoiding upstream model changes.
 - Feistel Cipher Shuffler:** Developed an $O(1)$ space and amortized- $O(1)$ time [Feistel cipher](#)-based bijective shuffler using cycle-walking, capable of shuffling a 402M+ example dataset without materializing the 1.6 GB permutation array.
 - Streaming Data Pipeline:** Built a streaming training pipeline over 402M+ examples (204 GB) of raw, unprocessed Parquet data—eliminating offline pre-processing and duplicate storage. On initialization, a background thread pre-fetches PyArrow row groups to local SSD in the exact order the Feistel-shuffled permutation will access them; zero-copy `pa.Table.slice()` extracts individual rows, which are JIT-tokenized and collated in a bounded prefetch queue. Achieved zero TPU starvation.
 - Shared-Prefix Sequence Packer:** Designed a sequence packing scheme where multiple training examples share a single system prompt prefix within one packed sequence. Implemented a star-pattern causal attention mask—each example attends to the shared prefix and itself but not to other examples—encoded as per-segment bitmasks and injected into JAX's Pallas splash attention kernel to avoid materializing dense $O(L^2)$ masks.
- Protopia AI** Remote, San Francisco, CA
Research Engineer May 2022–Jun 2025
 - Bounded-Memory KNN:** Implemented a memory-bounded K-nearest-neighbors operation over LLM embeddings by tiling `torch.cdist` along the sequence and vocabulary dimensions, reducing within each tile, and merging. Avoids materializing the full $[\text{seq_len}, \text{vocab_size}]$ distance matrix.
 - Distributed PyTorch Training Library:** Owned and rearchitected the company's PyTorch training library. Scaled training to Llama 3.1 405B using FSDP2 and Tensor Parallel. Integrated W&B logging and custom evaluation metrics.
 - Hook-Based Loss Framework:** Designed a composable loss function framework using PyTorch forward hooks to collect arbitrary intermediate tensors during the forward pass—without modifying model code—and route them into user-defined loss functions. Adopted as the standard loss API across all training workflows at the company.
 - Real-time Video Processing Framework:** Architected a real-time video processing framework using `multiprocessing.shared_memory` and OpenCV; implemented vectorized face-tracking in NumPy.
- Amazon.com, Inc.** Austin, TX
Software Development Engineer Jun 2020–May 2022

OPEN SOURCE

- Add padding_side to torch.utils.rnn.pad_sequence:** Added `padding_side` parameter to the C++ `pad_sequence` API, improving left-padded LLM input collation speed by 100%.
- Sharded Hugging Face model loading:** Extended `accelerate.load_checkpoint_in_model` to use `torch.distributed.checkpoint.state_dict.set_model_state_dict` for a 50% improvement in sharded model load time.
- Hugging Face transformers DDP Gradient Synchronization Bug:** Discovered and fixed a bug where PyTorch `DistributedDataParallel` was silently not synchronizing gradients for `transformers` models when using a static graph.
- Distributed CPU checkpoint loading on NCCL backend:** Modified `torch.distributed._state_dict_utils._broadcast_tensors` to automatically create communicated tensors on a device supported by the active `ProcessGroup`.
- First documented usage of FSDP and FSDP2 with Hugging Face generate:** Improved documentation, added automatic FSDP-wrapping detection to avoid communication deadlocks, added regression tests.
- einops-style rearrange in functorch:** Implemented `einops.rearrange` natively in `functorch`. Used the parsed string to dynamically build functions to perform the `rearrange` operation using [PyTorch first-class dimensions](#).
- Hugging Face datasets cache reuse across worker counts:** Refactored `Dataset.map` to discover and reuse cached shards from prior runs regardless of `num_proc`, eliminating redundant remapping when changing worker counts.

EDUCATION

- University of Texas at Austin** Austin, TX
B.S. Computer Science; GPA: 3.75/4.00 Aug 2015–May 2020